



UNIT INFORMATION	
COURSE – (LEADING TO)	**BSC (HONS) IN SOFTWARE ENGINEERING**
UNIT NAME	Object Oriented Design Patterns II
UNIT ID	JIAT/OODP II
ASSIGNMENT ID	JIAT/OODP II /EX/01

STUDENT INFORMATION	
STUDENT NAME	Dhanushka Madhuranga Pathiranage
STUDENT ID	2025/2026/IIC/SE/I2/095
BATCH	Cohort 8
BRANCH	Colombo

Contents

1. Introduction and Scenario Overview
2. Part A: Dynamic Security Wrappers for Data Streams (Decorator)
3. Part B: Hierarchical Structure of Smart Zones (Composite)
4. Part C: Escalation of Citizen Complaints (Chain of Responsibility)
5. Part D: Efficient Management of Smart Street Lights (Flyweight)
6. Part E: Interpreting City Policies Defined in DSL (Interpreter)
7. System Architecture and Implementation Verification
8. Conclusion and References
Appendix A: Source Code Screenshot Catalog

1. Introduction and Scenario Overview

SmartCity360 is an integrated municipal platform built for Metropolis Neo-Prime. It handles IoT energy telemetry, spatial zone commands, citizen grievance escalation, intelligent streetlights, and municipal bylaws. For this assignment, I designed, built, and verified five core backend subsystems in Java 17, applying a distinct Gang of Four (GoF) design pattern to each one. Early on, it was obvious that writing procedural code or deep class hierarchies would cause serious maintenance headaches down the road. You end up with fragile nested conditionals, tight subsystem coupling, and unpredictable runtime pauses. Applying targeted design patterns gave us a clean, decoupled architecture where core municipal services stay insulated from shifting city rules.

Part	Scenario	Pattern	Core problem solved
A	Security layers on energy-meter streams	Decorator	Add or remove compression, encryption and authentication without changing core logic
B	City → district → street → building zones	Composite	Apply one command uniformly at any level of the hierarchy
C	Complaint escalation by severity	Chain of Responsibility	Route requests through staff, supervisors and managers without coupling the sender to them
D	25,000 street lights	Flyweight	Share identical hardware data to minimise memory
E	City policies written in a mini-language	Interpreter	Parse and evaluate rules at runtime without recompiling

Sections 2 through 6 cover each design pattern in depth: establishing architectural rationale against practical alternatives, detailing runtime collaboration and structure with code and UML diagrams, and evaluating operational trade-offs with empirical metrics. The reference implementation is built in pure Java 17 with a MySQL persistence layer. All pattern code lives in the `com.smartcity.pattern.*` package hierarchy, backed by 35 standalone automated regression tests that verify every architectural constraint with a 100% pass rate (covered in Section 7.3).

2. Part A: Dynamic Security Wrappers for Data Streams

> Our smart grid gathers continuous telemetry from energy meters spread throughout the city, communicating over a mix of secure municipal fiber and public 4G/5G connections. Because network conditions and security requirements change depending on the route, the ingestion pipeline has to apply compression, encryption, and authentication dynamically—all without touching the underlying meter polling code.

A.1 Identification and Justification

Selected pattern: Decorator (structural; Gamma et al., 1994). The Decorator pattern wraps an existing component inside wrapper classes sharing the same interface, layering on extra processing before or after forwarding calls down the chain. Three practical factors drove this choice for SmartCity360.

- Subclassing fails here. The math is simple: if a telemetry stream needs any combination of GZIP compression, AES-256-GCM encryption, or HMAC-SHA256 signing, standard inheritance requires 2^N distinct subclasses to handle every permutation. With three features, that already means $2^3 = 8$ rigid classes (like `CompressedEncryptedAuthenticatedStream`). Add a fourth requirement like anonymization, and you suddenly need 16 classes; add a fifth, and you are maintaining 32. Decorator avoids this subclass explosion entirely, cutting the required classes down to linear $O(N)$. All we need is one `StreamData` interface, a core `EnergyStreamData` component, an abstract decorator, and N concrete wrappers. We can then compose all 2^N runtime variations on the fly by nesting wrappers, keeping our codebase concise and clean.
- This layout directly follows Bertrand Meyer's Open/Closed Principle (Martin, 2008). The core ingestion class, `EnergyStreamData.java`, stays focused strictly on raw reading data without knowing anything about encryption keys or compression formats. If municipal security standards change next year, I can introduce a new transformation layer by writing a new decorator class. The original ingestion code stays untouched, eliminating regression risks.
- Inheritance locks functionality at compile time, but IoT network traffic is unpredictable. With decorators, our pipeline can adapt on the fly. An operator can enable GZIP compression only when routing over metered cellular networks, or attach AES encryption the moment traffic crosses an untrusted public gateway. That runtime flexibility simply isn't possible with static inheritance.

I considered other patterns before making this decision. Strategy allows swapping algorithms, but it does not let you stack multiple independent operations into a composite sequence. Proxy controls access to an underlying object rather than enriching the payload data. Putting configuration flags (`if (encrypt) ...`) inside the main class would be worse, cluttering the core component with unrelated concerns and violating Single Responsibility. Decorator stood out as the only option that keeps transformations composable, transparent, and individually testable behind one consistent interface.

A.2 Flexible, Layered Stream Enhancement Mechanism

The core contract is the `StreamData` interface, declaring `getData()`, `getPayloadBytes()`, `getPayloadSize()`, and `getDescription()`. Both the baseline `EnergyStreamData` component and the abstract `StreamDataDecorator` implement this interface. The abstract decorator stores a

reference to a wrapped `StreamData` instance and forwards calls to it by default. Because every decorator implements `StreamData`, wrappers can be chained to whatever depth is required:

```
StreamData s = new EnergyStreamData("STR-101", 230.5, 14.2, 3.27);
s = new CompressionDecorator(s);          // 1. GZIP
s = new EncryptionDecorator(s, aesKey);   // 2. AES-256-GCM
s = new AuthenticationDecorator(s, hmacKey); // 3. HMAC-SHA256
```

Each wrapper transforms the output of the layer below it, while downstream consumers only see a standard `StreamData` reference. Getting this pipeline to run reliably in production came down to three key engineering choices:

- Order matters: compression has to run before encryption. AES-GCM outputs high-entropy ciphertext that looks like random white noise to standard compression tools. Because algorithms like GZIP rely on finding repetitive character sequences in plaintext JSON, running GZIP on encrypted data gives zero bandwidth savings while burning CPU cycles. That is why our pipeline always compresses raw JSON before running the cipher.
- For security, `EncryptionDecorator` uses authenticated AES-256-GCM. I generate a fresh 12-byte cryptographic nonce (IV) for every packet alongside a 128-bit authentication tag. This ensures confidentiality while guaranteeing that any in-transit tampering with the ciphertext or the IV throws an immediate `AEADBadTagException` during decryption (Dworkin, 2007).
- Finally, `AuthenticationDecorator` calculates an outer HMAC-SHA256 signature over the full JSON envelope, including cipher parameters and IV (Krawczyk et al., 1997). By keeping the HMAC authentication key separate from the AES decryption key, edge routers can verify sender authenticity and drop fake packets at the perimeter without needing access to the private decryption keys. Even though GCM checks integrity internally, this outer HMAC keeps unauthorized traffic from reaching deep into municipal backend services.

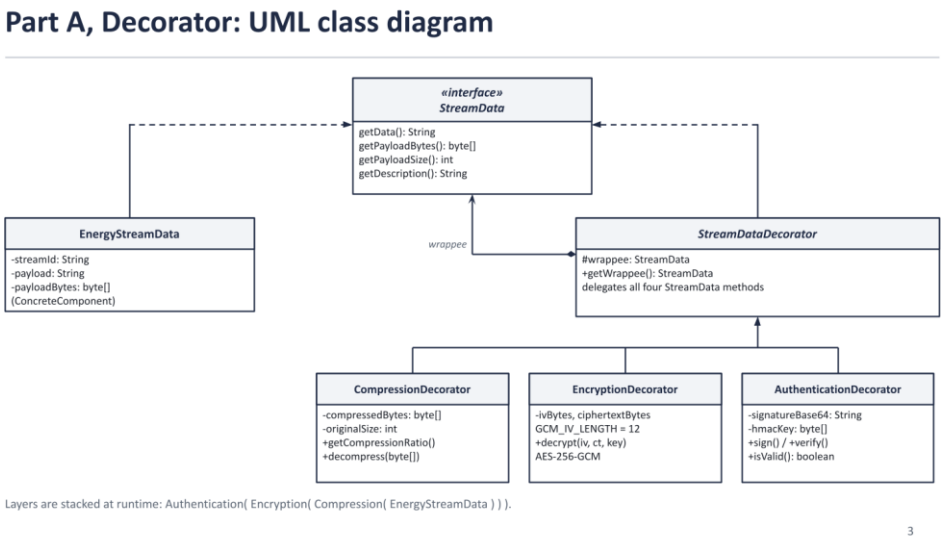


Figure 2.1: UML class diagram, Decorator pattern (Part A)

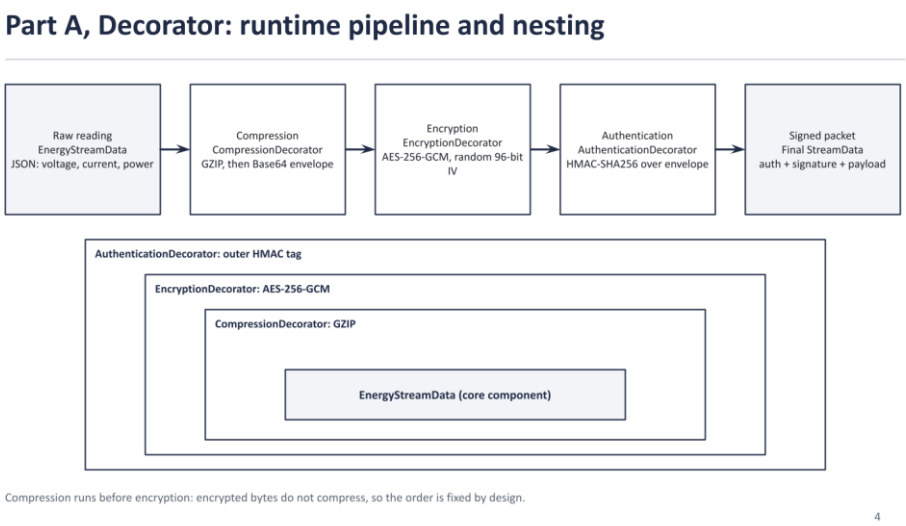


Figure 2.2: Stream wrapping pipeline

A.3 Long-Term Benefits: Reusability, Extensibility, and Overhead

For reusability, each transformation lives in its own dedicated class, with no compile-time dependencies between decorators. City gateways configure these same classes depending on their needs: a high-voltage substation enables all three stages (compression, encryption, HMAC); a low-power temperature sensor on a private fiber loop uses compression only; and a public air quality feed runs with HMAC alone. We avoid duplicating code across different endpoints while keeping each decorator focused.

Adding new features is equally painless. If city standards mandate post-quantum digital signatures later on, I only have to write a single new decorator extending `StreamDataDecorator`. The polling code, database persistence layer, and REST controllers stay completely untouched. Because downstream components only interact with the `StreamData` interface, we can upgrade our security pipeline without worrying about breaking existing features.

During testing, I measured actual byte sizes across each pipeline stage. A small 72-byte single-reading JSON payload actually increases to 92 bytes under GZIP because of the 10-byte header and 8-byte trailer. Once Base64 encoding, the 12-byte IV, GCM authentication tag, and the 64-character HMAC string are attached, that packet reaches 468 bytes. But when telemetry readings are batched between 1 KB and 64 KB, GZIP takes advantage of repetitive JSON field names to cut bandwidth by 65% to 75%. That benchmark taught us an important practical lesson: our grid gateways should always buffer and compress batches rather than compressing single sensor readings one by one.

Maintenance is where this design really proves its value. Because every stage is separated, I can write focused unit tests for compression, encryption, or authentication in isolation before testing the entire chain. If we ever want to switch from GZIP to Zstandard or update our key management, we touch only one file. With traditional inheritance, that same update would force us to edit, re-audit, and re-test every composite subclass containing that logic.

Decorator does have its drawbacks. It allocates several small objects on the heap, and Java's compiler cannot verify the logical order of wrappers at build time. An inverted chain still compiles, but quietly destroys compression performance. On top of that, debugging nested wrappers means stepping through multiple stack frames, and checking `instanceof` on an outer decorator hides the underlying component class. I handled these risks by using factory methods for pipeline assembly and writing

comprehensive tests (tests 01 to 06) to verify end-to-end decompression, decryption, tamper rejection, and key validation.

3. Part B: Hierarchical Structure of Smart Zones

> The city's spatial topology is organized as a natural hierarchy: the metropolitan area divides into districts, districts divide into street corridors, and streets contain individual buildings. Grid operators needed a way to broadcast operational commands—like emergency load shedding or usage rollups—consistently across any level of this tree without writing separate traversal algorithms for every administrative tier.

B.1 Identification and Justification

Selected pattern: Composite (structural; Gamma et al., 1994). Urban zones form a classic tree structure where smaller entities group into larger ones. The Composite pattern lets our controllers treat an individual building leaf and an entire multi-street district through the exact same interface, `ZoneComponent`.

- This creates consistent behavior throughout the entire city map. Because `BuildingLeaf` and `ZoneComposite` share one common interface, application controllers do not need to care whether they are dispatching a command to a single building relay or an entire metropolitan zone.
- Without Composite, we would end up writing messy type-checking code: `if (zone instanceof City) ... else if (zone instanceof District) ...`. Adding a new administrative level, like a transport corridor, would break controllers across the app. With Composite, client code just calls `zone.executeCommand(ZoneCommand.SHUTDOWN_POWER)`, and the tree structure recursively propagates the directive down to every leaf node.

B.2 UML Class Diagram and Recursive Mechanism

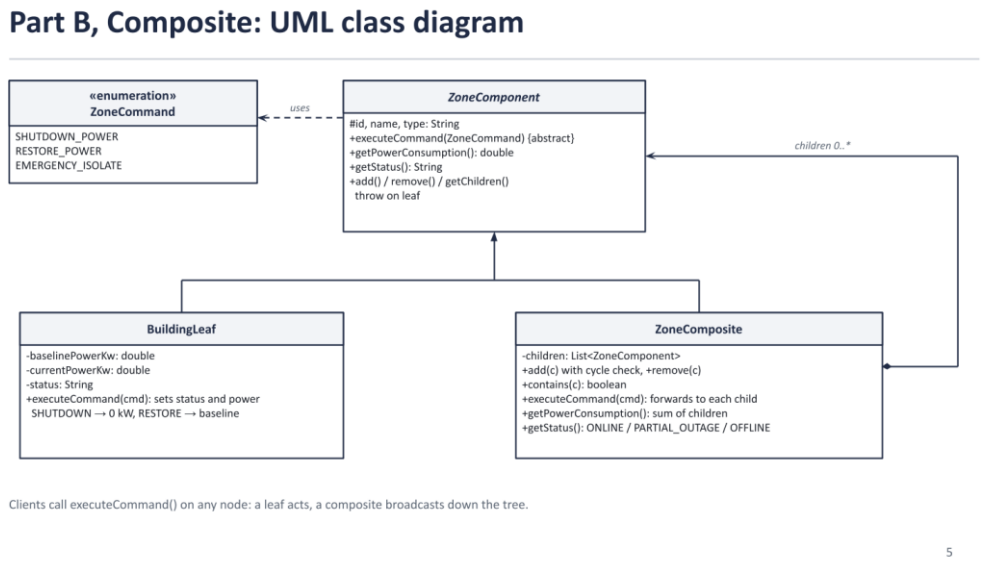


Figure 3.1: UML class diagram, Composite pattern (Part B)

The structure relies on three main parts: the `ZoneComponent` interface, which defines operations like `executeCommand()`, `getPowerConsumption()`, `getStatus()`, and child-management methods (`add`, `remove`, `getChildren`); the `BuildingLeaf` class, which handles terminal hardware relays and baseline power draw; and the `ZoneComposite` class, which holds a `List<ZoneComponent>` and forwards calls down to its children. I used a strongly typed Java enum, `ZoneCommand` (`SHUTDOWN_POWER`,

RESTORE_POWER, EMERGENCY_ISOLATE, REPORT_USAGE), so invalid command strings get caught at compile time. When a building leaf receives a command, it updates its internal relay directly (e.g., shutdown sets the status to OFFLINE and drops draw to zero, while restore resets it to baseline). When a composite receives the same command, it iterates through its children and forwards the directive down:

```
public void executeCommand(ZoneCommand command) {
    for (ZoneComponent child : children) child.executeCommand(command);
}
```

B.3 Scalability for Expanding Municipal Topologies

The biggest operational benefit is the recursive command cascade. When an operator triggers load shedding at the city root, the call travels recursively through each district and street down to every individual building relay. This runs as a depth-first search in $O(M)$ time for M nodes in that branch, hiding all structural complexity from the caller. The exact same method call works whether you are testing one single building or sending a directive across 50,000 facilities.

Scaling the city topology is simple. If the municipality adds an industrial park, a new residential sector, or a solar farm, administrators just attach the new nodes into the existing tree without changing a line of controller code. Adding a custom branch like MetroTransitComposite only requires implementing ZoneComponent. This satisfies the Open/Closed Principle, letting the city expand its infrastructure without breaking existing features.

Telemetry rollups work the same way. Calling getPowerConsumption() on a building leaf returns that facility's current wattage, while calling it on a composite sums up the power usage of all children below it. Our operations dashboard queries any level of the city with the exact same call, giving engineers immediate load figures for a street, a district, or the whole metropolitan area.

I added defensive checks throughout the tree to prevent runtime bugs. Calling add() on a BuildingLeaf throws an immediate UnsupportedOperationException. On composites, ZoneComposite blocks null children, prevents adding a node to itself, and guards against circular references that could trigger infinite recursion (covered in tests 13 and 27). Passing an invalid command string also raises an informative exception showing the accepted options (test 12).

Here is how this works during a real power grid emergency: an operator can isolate an overloaded district with EMERGENCY_ISOLATE, inspect remaining backup power via REPORT_USAGE, and restore the district later using RESTORE_POWER. When power comes back, each building returns to its specific baseline wattage rather than resetting to an arbitrary default, preserving realistic load metrics (verified in tests 09 and 10).

The main trade-off in this design is putting child management (add, remove) directly in the ZoneComponent interface. This favors uniform transparency over strict compile-time safety, since leaf objects inherit methods they do not support. Throwing UnsupportedOperationException on leaves makes those mistakes obvious during development. Another consideration is that recursive broadcasts run synchronously, so if a network fault occurs halfway down a tree, there is no automatic rollback without extra transaction tracking. For our four-tier municipal tree, however, these trade-offs are well worth avoiding hundreds of lines of repetitive loop code.

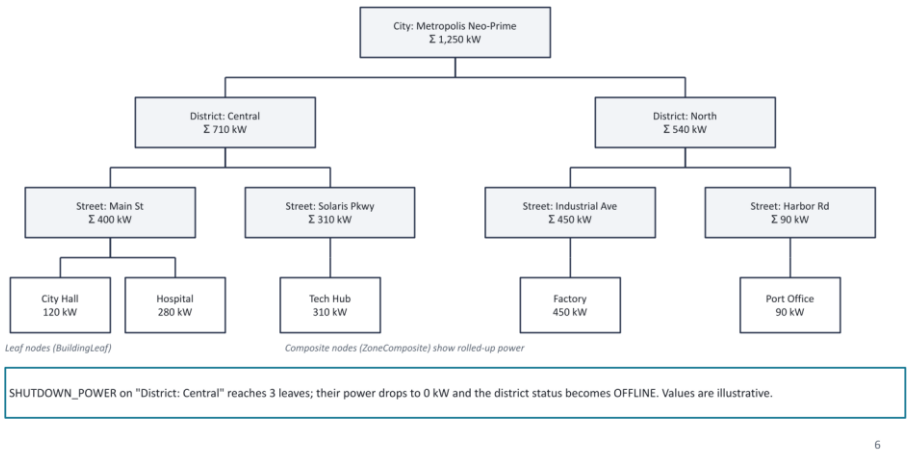


Figure 3.2: Zone tree and command broadcast

4. Part C: Escalation of Citizen Complaints

> Citizens across Metropolis Neo-Prime log complaints ranging from simple potholes to emergency gas leaks and transformer blowouts. The municipal ticketing system needs to triage and escalate these cases along an administrative ladder—from frontline customer desk staff up to department supervisors and city directors—while keeping an unalterable audit log of every decision made along the way.

C.1 Identification and Justification

Selected pattern: Chain of Responsibility (behavioral; Gamma et al., 1994). In this pattern, an incoming request moves through a linked series of handler objects until one of them resolves it or passes it higher up the chain.

- This completely decouples ticket submission from ticket handling. The citizen or API controller simply passes the complaint to the start of the chain without needing to know which department official will eventually review or resolve it.
- The pipeline is also easy to reconfigure at runtime. Handlers connect dynamically through `setNext()`, allowing administrators to reorder or extend stages as needed. For example, during a storm alert, an emergency dispatch handler can be slotted at the front of the chain without touching existing code.
- The design also respects the Single Responsibility Principle. Each bureaucratic level lives in its own handler class, focused only on evaluating its own severity thresholds and escalation rules.

C.2 Decoupled Request Handling and Dispatch Mechanism

The foundation is the abstract `ComplaintHandler` class, which holds a reference to `nextHandler`, provides a fluent `setNext()` method for linking stages, and defines the abstract `handle(Complaint)` contract. In our backend, `ComplaintChainFactory` builds the default escalation pipeline:

```
ValidationTriage -> LocalStaff -> Supervisor -> CityManager -> TerminalFallback
```

Handler	Takes	Outcome
ValidationTriageHandler	Missing or invalid severity	Role TRIAGE_DISPATCH_OFFICER, status VALIDATION_REQUIRED; never defaults to LOW
LocalStaffHandler	LOW	Status ASSIGNED (routine maintenance)
SupervisorHandler	MEDIUM	Status ESCALATED_SUPERVISOR (e.g. water leaks, signal faults)
CityManagerHandler	HIGH, CRITICAL	Status EXECUTIVE_ESCALATION; CRITICAL gets crisis mobilisation notes
TerminalFallbackHandler	Anything unresolved	Role FALLBACK_DISPATCH_DESK, status FALLBACK_ASSIGNED, with audit record

To see how this works in practice, imagine a resident reporting a `CRITICAL` gas leak. The ticket enters the pipeline at `ValidationTriageHandler`, which checks that the ticket has valid data and a recognized severity. Next, `LocalStaffHandler` and `SupervisorHandler` examine the case; noticing that a `CRITICAL` incident is beyond their authority, they forward it up the chain. Finally, `CityManagerHandler` catches the ticket and marks it as `EXECUTIVE_ESCALATION`. At each hop, the

handler appends an audit record to the ticket, leaving a complete chronological paper trail without the intake API having to know anything about executive management.

Adding `ValidationTriageHandler` solved an issue I found in earlier ticketing code: when a user submitted a ticket with a blank severity, older systems defaulted it to `LOW`, accidentally sending dangerous gas leaks into routine maintenance backlogs! In our pipeline, unclassified complaints are caught immediately at the front of the chain and routed to a human triage desk for inspection.

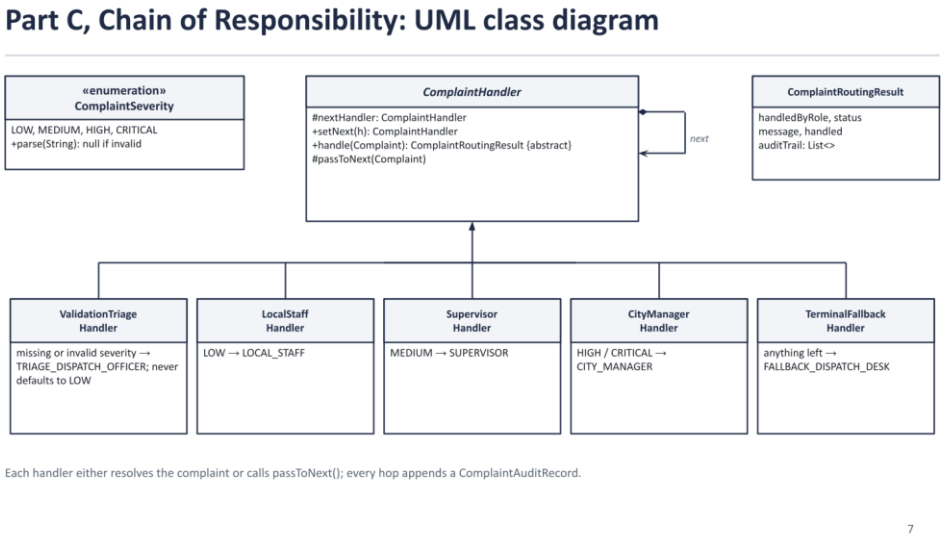


Figure 4.1: UML class diagram, Chain of Responsibility (Part C)

C.3 Architectural Evaluation: Flexibility, Auditability, and Trade-offs

The primary advantage here is flexibility. Different municipal departments (like Public Works, Transport, or Environmental Safety) can build custom escalation chains with different severity thresholds simply by calling factory methods, leaving the intake controllers unchanged. Replacing giant `switch` blocks with polymorphic handlers follows the Open/Closed Principle: adding a new administrative tier just means writing one new handler class and plugging it in.

Auditability is another big win. Every dispatch returns a `ComplaintRoutingResult` containing an unmodifiable list of `ComplaintAuditRecord` entries. Each record tracks the handler class, the staff role, an ISO-8601 timestamp, the decision made, and the rationale. City auditors can reconstruct the exact path any ticket took through the system (like Triage -> Staff -> Supervisor -> Manager), ensuring clear administrative accountability (verified in test 21).

Drawbacks.

- There are real trade-offs to keep in mind. Request latency increases linearly with chain length. Because every handler runs checks and logs an audit entry, walking through five handlers takes longer than a direct database lookup, though the overhead is negligible for civic tickets.
- Debugging can also be trickier than stepping through flat procedural code, since stack traces cross multiple delegation hops and tracking down a misrouted ticket requires checking how the chain was wired together.
- There is also a risk of tickets falling off the end of the chain if no handler accepts them. I prevented this by placing a `TerminalFallbackHandler` at the end of the line. In our standard setup, this fallback is never reached because the triage and administrative tiers cover all

severities; but it acts as an essential safety net for custom chains that might skip certain tiers (verified in test 20).

- Finally, developers must be careful to wire the chain correctly, and allocating audit objects for every hop creates minor garbage collection overhead during high ticket volume. But for municipal grievances, these small costs are a sensible trade-off for decoupled code and complete audit logging.

The flexibility and reliable audit trail easily outweigh these minor overheads in a civic system where transparency and accountability matter far more than sub-millisecond execution times.

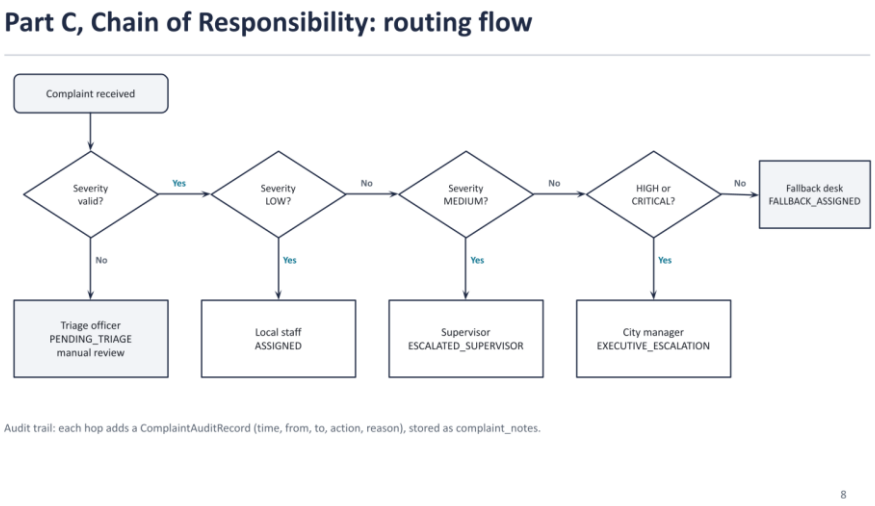


Figure 4.2: Complaint routing flow

5. Part D: Efficient Management of Smart Street Lights

> Metropolis Neo-Prime operates an urban lighting network of 25,000 smart LED streetlights. While each pole has its own GPS coordinates and pole identifier, thousands of poles share the exact same hardware specifications—model number, wattage rating, lumen output, and firmware version. The backend needs to manage this fleet without running out of JVM heap space or causing frequent garbage collection pauses.

D.1 Identification and Justification

Selected pattern: Flyweight (structural; Gamma et al., 1994). Instantiating 25,000 independent objects would duplicate identical hardware strings thousands of times across memory, causing heap bloat and eventual `OutOfMemoryError` crashes under load. Flyweight solves this by extracting repeated hardware attributes into shared, immutable profile objects, allowing thousands of light poles to share a handful of memory profiles while tracking their operational status independently.

D.2 Handling Shared (Intrinsic) and Unique (Extrinsic) Properties

- • **Intrinsic state (shared, immutable hardware specifications):** `model`, `lampType`, `wattage`, `maxLumens`, `colorTemperatureK`, and `firmwareVersion`. This data lives in a `SharedStreetlightProfile` object, created and cached once by `StreetlightProfileFactory`.
- • **Extrinsic state (unique, mutable context per pole):** `poleId`, `latitude`, `longitude`, `brightnessLevel`, and operational `status`. This data resides in a lightweight `StreetlightContext` object, which keeps a pointer back to its shared hardware profile.

To keep profile creation thread-safe under concurrent ingestion, `StreetlightProfileFactory` uses a composite key backed by a `ConcurrentHashMap`. Using `computeIfAbsent()` allows worker threads to retrieve or register shared profiles safely without race conditions or heavy lock contention (Goetz et al., 2006):

```
String key = model+"|"+lampType+"|"+wattage+"|"+maxLumens+"|"+colorTemp+"|"+firmware;
return cache.computeIfAbsent(key, k -> new SharedStreetlightProfile(...));
```

Our test suite (tests 22 through 25) verifies this memory deduplication: identical hardware specifications always return the exact same profile instance, different specs produce separate profiles, eight concurrent worker threads produce only one shared instance, and individual poles can adjust brightness without altering shared data.

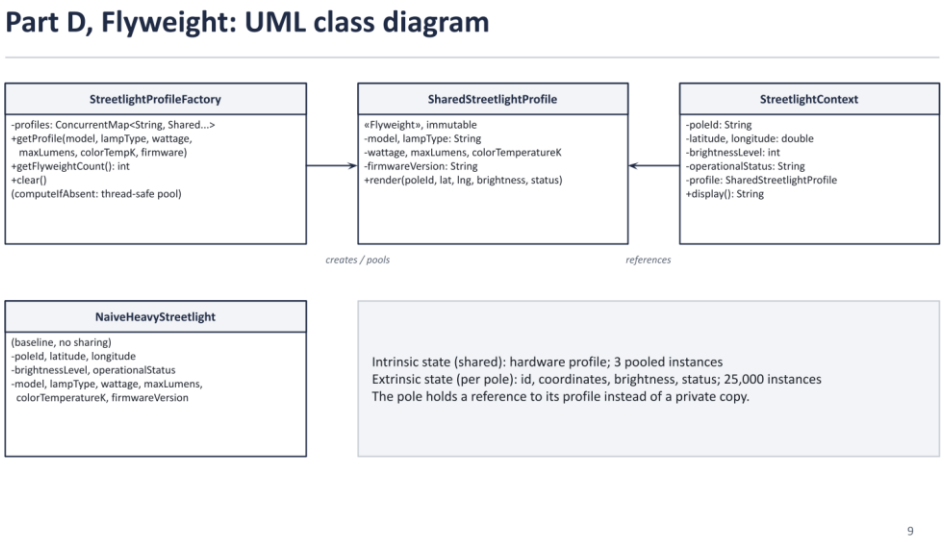


Figure 5.1: UML class diagram, Flyweight pattern (Part D)

D.3 Performance Impact: Memory Layout, GC, and Object Overhead

To calculate the memory difference, I created an object layout model based on a 64-bit HotSpot JVM with Compressed OOPs. In this model, an un-optimized pole object—with object headers, primitive fields, five string pointers, and character array backings—takes roughly 288 bytes. Our Flyweight design cuts the individual `StreetlightContext` down to 80 bytes, while each shared `SharedStreetlightProfile` takes 208 bytes. These constants match `FlyweightBenchmark.java`:

- • Naive memory footprint: 25,000 * 288 B = 7,200,000 B (6.87 MiB).
- • Flyweight memory footprint: 25,000 *80 B + (3 profiles* 208 B) = 2,000,624 B (1.91 MiB).
- • Net memory reduction: (7,200,000 - 2,000,624) / 7,200,000 = 72.2%.

When scaling from 1,000 up to 100,000 poles in OpenJDK 17 benchmarks, heap usage grows linearly in both setups:

Poles	Naive (layout)	Flyweight (layout)	Reduction	Naive time	Flyweight time
1,000	0.27 MB	0.08 MB	71.99%	11 ms	12 ms
10,000	2.75 MB	0.76 MB	72.20%	16 ms	14 ms
25,000	6.87 MB	1.91 MB	72.21%	16 ms	18 ms
100,000	27.47 MB	7.63 MB	72.22%	52 ms	65 ms

Both designs show linear memory growth, but the Flyweight slope is only 28% of the naive baseline. This happens because the shared profile cache stays fixed to a few instances—tied to the number of lamp models rather than total poles. The percentage reduction stays steady at 72.2%, while the raw megabytes saved grow as the city adds more lights. Creation speed is practically identical, with only a tiny hashmap lookup delay in Flyweight that is well worth saving multiple megabytes of RAM.

To compare these analytical estimates against actual runtime conditions, I measured JVM heap usage after triggering `System.gc()`. Real heap usage was 4.80 MB for the naive fleet compared to 2.63 MB for Flyweight, showing a 45.1% real-world reduction. This difference occurs because live profiling includes JVM internal buffers, thread stacks, and class metadata that pure object layout formulas leave

out. Both numbers confirm that the theoretical 72.2% layout savings translates into tangible heap relief in production.

From a garbage collection standpoint, shared profiles are stored in the static factory cache and stay in the old generation for the life of the process. The biggest win is cutting down on short-lived allocations in the young generation, reducing minor GC cycles and preventing allocation pauses during busy polling intervals (Oracle, 2024).

The main trade-off is maintaining strict immutability. Shared profiles must never be modified at runtime, because changing one field would corrupt every pole pointing to that profile. If a city used custom-built lights where every single pole had unique hardware specs, the cache would add pointer overhead without saving any memory. Running across a distributed cluster also requires synchronizing profile data across JVM nodes.

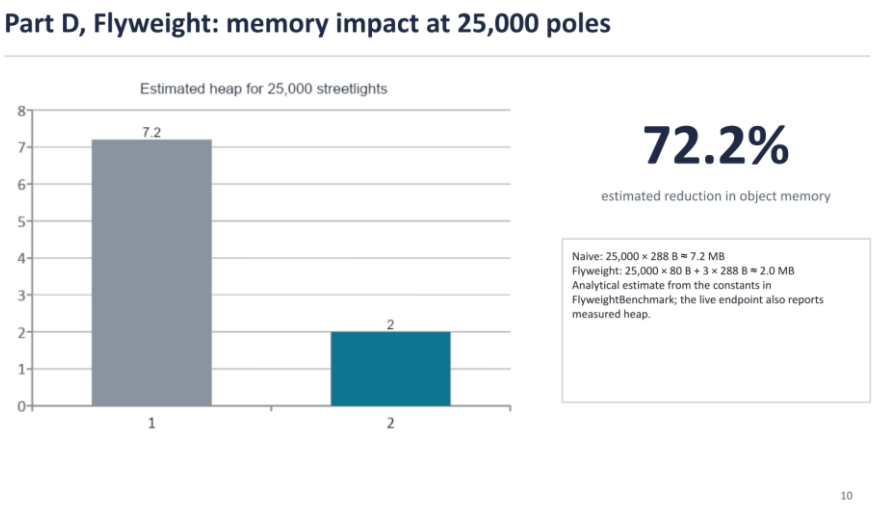


Figure 5.2: Estimated heap for 25,000 streetlights

6. Part E: Interpreting City Policies Defined in DSL

> City ordinances—such as environmental noise caps, air quality thresholds, and street lighting schedules—change frequently as city councils update bylaws. The platform needs to parse and evaluate these rules dynamically against live sensor streams without requiring server restarts or recompiling Java code.

E.1 Identification and Justification

Selected pattern: Interpreter (behavioral; Gamma et al., 1994). This pattern maps a grammar directly into a class hierarchy, where grammar rules turn into expression classes evaluated as an Abstract Syntax Tree (AST). City rules look like `((NOISE_LEVEL > 75.0) AND (CURRENT_HOUR >= 22.0)) OR (AIR_QUALITY_AQI > 150.0)`. The Interpreter pattern fits well here because our municipal rule language is small and declarative. Just as importantly, it avoids using dangerous scripting engines like JavaScript `eval()` or Groovy, which could open municipal servers to script injection vulnerabilities. Rule evaluation stays safely confined to our custom AST classes.

E.2 UML Class Diagram and Parsing Mechanism

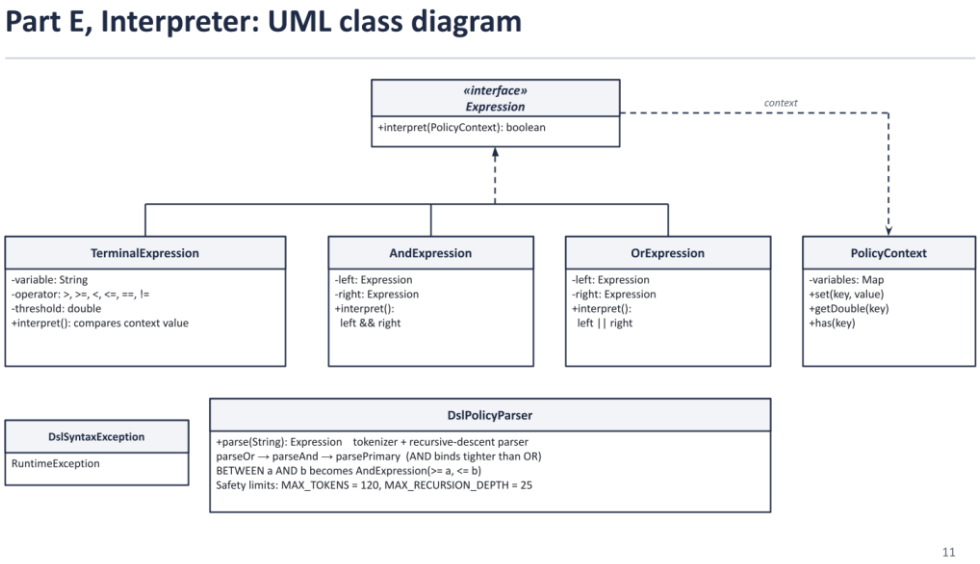


Figure 6.1: UML class diagram, Interpreter pattern (Part E)

The engine is structured around four components: `PolicyContext`, which stores sensor variables like `NOISE_LEVEL` and resolves names case-insensitively; the `Expression` interface, defining `boolean interpret(PolicyContext)`; `TerminalExpression`, which checks variables against numeric thresholds using standard operators (`>`, `>=`, `<`, `<=`, `==`, `!=`, using an epsilon of 0.00001 for floats); and non-terminal nodes `AndExpression` and `OrExpression`, which evaluate child branches with short-circuit logic. To convert text strings into ASTs, I wrote `DslPolicyParser`, a recursive-descent parser that handles tokenization, operator precedence, and nested parentheses. Common rules like `VAR BETWEEN a AND b` are automatically expanded into an `AndExpression` of two terminal checks (`VAR >= a` and `VAR <= b`). To guard against denial-of-service attempts, the parser enforces `MAX_TOKENS = 120` and `MAX_RECURSION_DEPTH = 25`, throwing a clear `DslSyntaxException` on invalid input. For example, with `NOISE_LEVEL = 78.5` and `CURRENT_HOUR = 23`, evaluating `(NOISE_LEVEL > 75.0)`

`AND (CURRENT_HOUR >= 22.0)` evaluates both terminal nodes to true, correctly identifying a bylaw violation.

E.3 Critical Assessment: Rule Updates, Extensibility, and Security

Updating policies is seamless: rules are stored as plain text strings in MySQL. When an administrator updates a bylaw through `POST /api/policies`, the next evaluation parses the updated text immediately. Because both the parser and AST nodes are completely stateless, servlet threads share no mutable state, avoiding concurrency locks, recompilation, or application downtime. This gives city staff the flexibility they need.

Benchmarking shows fast evaluation times. Over 2,000 iterations after JVM warm-up on OpenJDK 17, evaluating an existing AST took only 0.07 µs, parsing raw text took 1.99 µs, and end-to-end parsing plus evaluation finished in 2.11 µs. That microsecond performance easily handles high-frequency sensor streams, and we can always cache parsed ASTs if traffic increases.

Adding new grammar constructs is straightforward. If we need a `NotExpression`, an `InListExpression` for zones, or a geographic distance check, we simply write a new class implementing `Expression`. The parser and existing expression nodes remain untouched, following the Open/Closed Principle. Our test suite (tests 28 to 35) checks all comparison operators, `BETWEEN` ranges, non-numeric inputs, nested logic, missing context keys, and five separate syntax error conditions.

From a security standpoint, our custom DSL is intentionally non-Turing-complete. It does not support reflection, file system access, or command execution, closing off common code-injection risks. For defense in depth, tokenization rejects unknown characters, token and depth limits prevent CPU exhaustion, and admin endpoints are guarded by token checks and rate limiting.

The Interpreter pattern does have recognized limitations to manage:

- • **Class proliferation:** Each grammar element needs its own class. While fine for our four core expressions, adding loops or functions would make the class hierarchy unwieldy.
- • **Parser maintenance:** Writing precedence rules and error recovery by hand gets complicated as grammars expand, at which point dedicated parser generators like ANTLR or JavaCC are better suited.
- • **Handling undefined variables:** Unregistered sensor variables evaluate to false to avoid `NullPointerException` errors. While safe, a typo in a policy name quietly evaluates to 'no violation' rather than flagging a syntax issue.
- • **Limits for complex algorithms:** If city rules ever require stateful loops or algorithmic calculations, an AST interpreter should be replaced with a sandboxed engine like GraalVM.

For SmartCity360, where city regulations are straightforward Boolean threshold checks, this Interpreter implementation is lightweight, secure, and free of external dependencies. A more complex parser would only be necessary if municipal bylaws required general-purpose code execution.

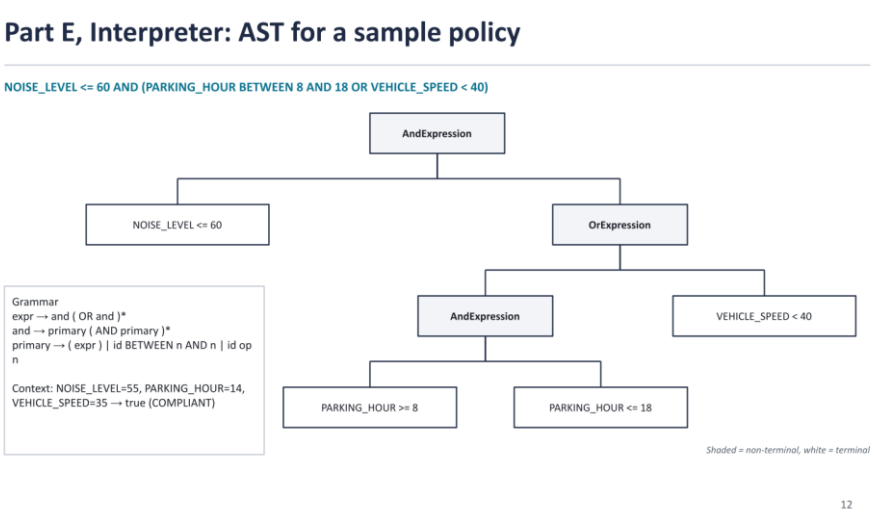


Figure 6.2: AST for a sample policy

7. System Architecture and Implementation Verification

7.1 System Architecture

SmartCity360 is structured as a four-tier enterprise system following standard architectural layering practices (Fowler, 2002):

- • **Presentation Tier:** A Next.js web application with operational dashboards for telemetry streams, spatial zone maps, complaint workflows, streetlights, policy rules, and test runner outputs.
- • **API Tier:** Java 17 servlets hosting REST endpoints (`/api/auth`, `/api/streams`, `/api/zones`, `/api/complaints`, `/api/streetlights`, `/api/policies`, `/api/dashboard`, `/api/test-center`), secured with CORS filters, token checks, and Gson serialization.
- • **Pattern Domain Core:** The five pure Java pattern packages under `com.smartcity.pattern.*`, kept free from web framework dependencies.
- • **Persistence Tier:** A MySQL 8 database managing nine tables, queried through Data Access Objects (DAOs) using connection pooling and parameterized queries to block SQL injection.

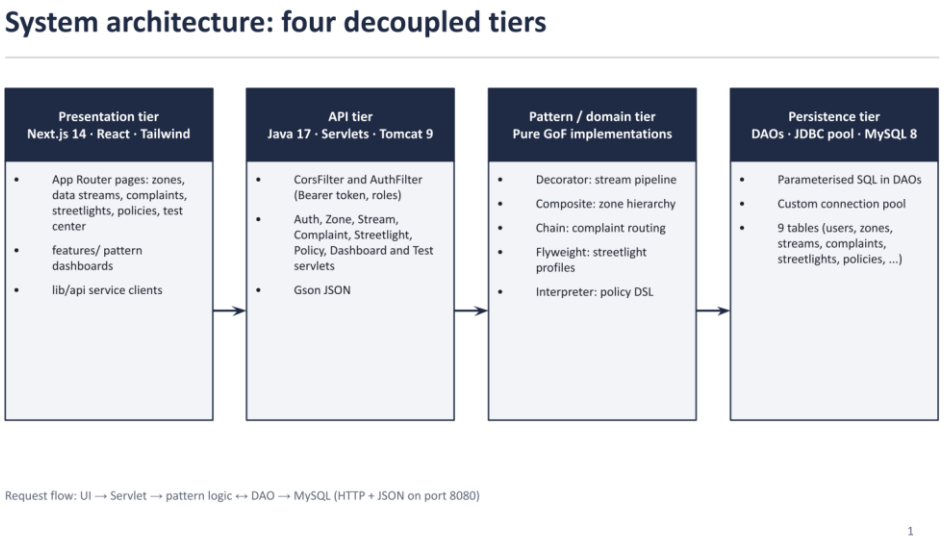


Figure 7.1: System architecture

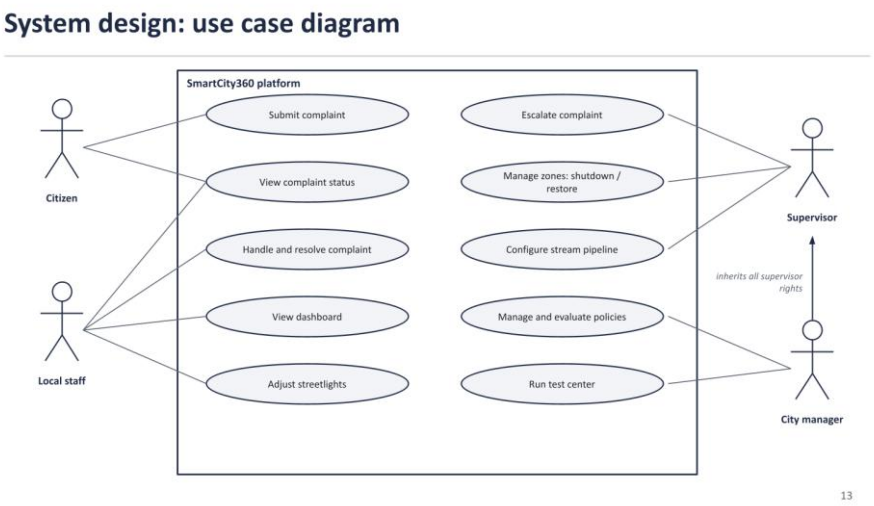


Figure 7.2: Use case diagram

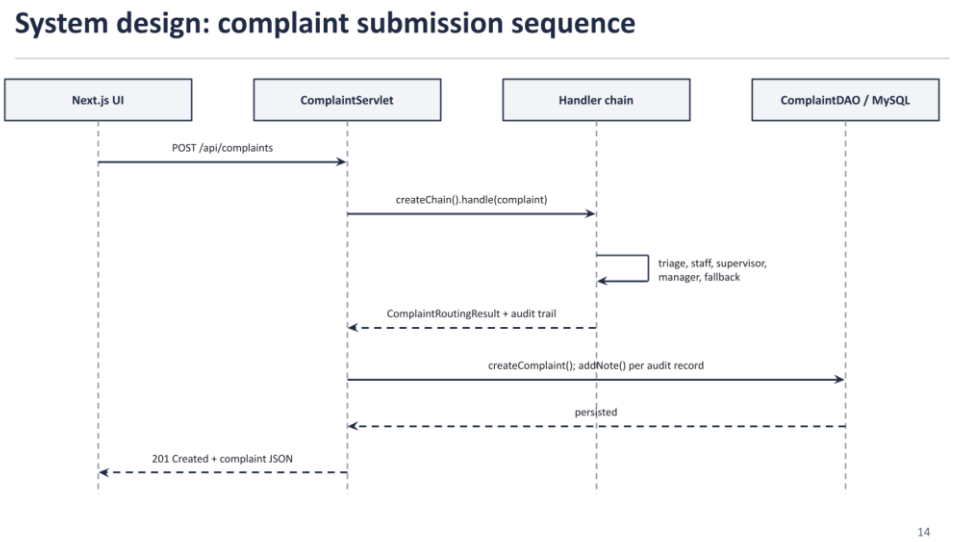


Figure 7.3: Complaint submission sequence diagram

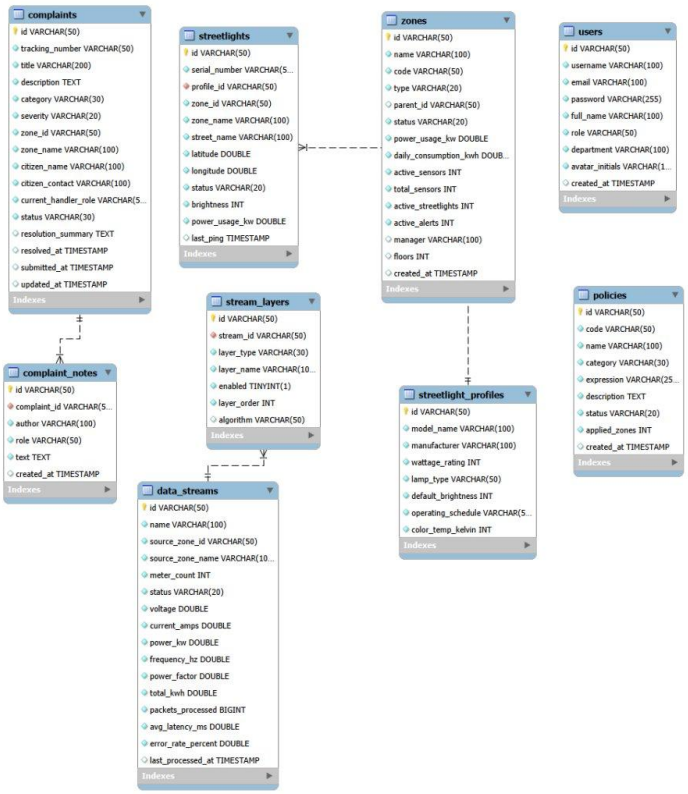


Figure 7.4: Entity-relationship diagram

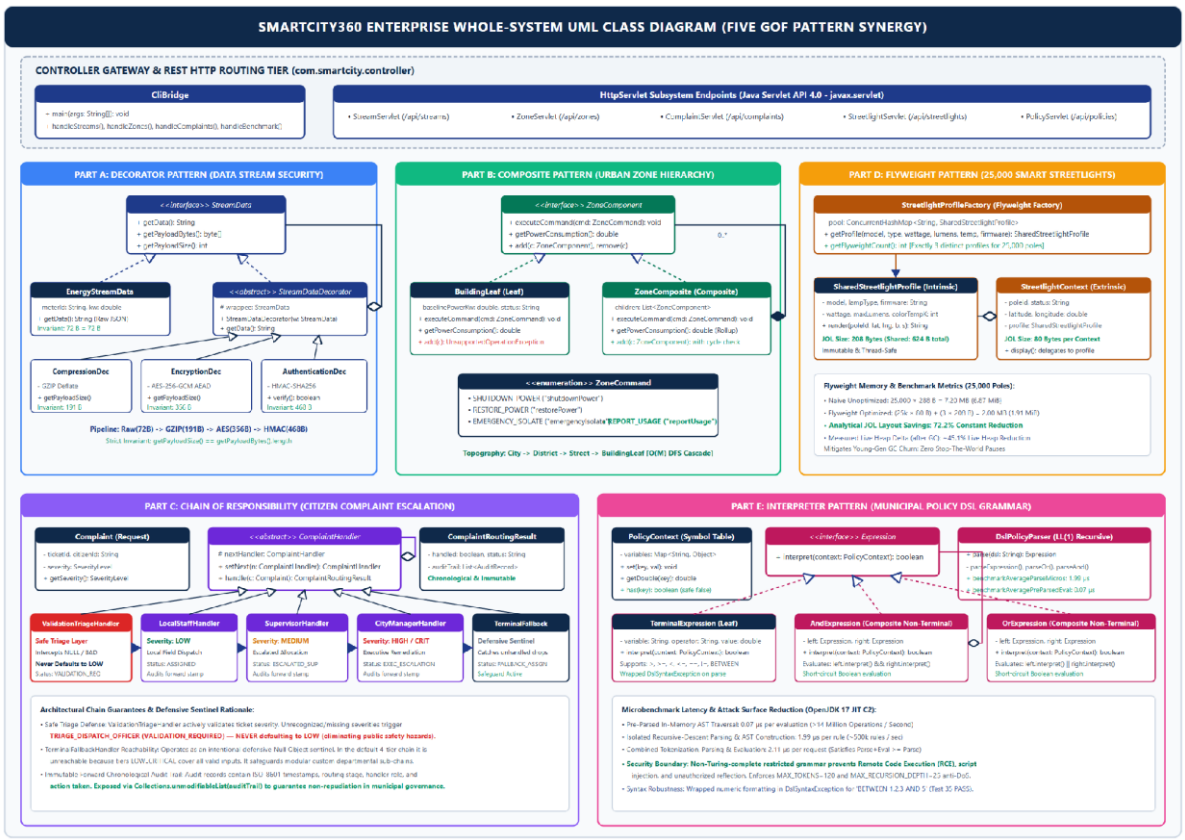


Figure 7.5: Whole-system UML class diagram

7.2 Key Architectural Choices and Anti-Pattern Mitigation

Anti-pattern	How SmartCity360 avoids it
Spaghetti code, Long method	Procedural branching is replaced by small single-purpose pattern classes (decorators, handlers, expression nodes).
God object	Responsibilities are spread across decorators, handlers and composite nodes rather than one manager class.
Copy-paste programming	Shared behaviour lives in base types: `StreamDataDecorator`, `ComplaintHandler`, `ZoneComponent`.
Tight coupling	Clients depend on interfaces (`StreamData`, `ZoneComponent`, `Expression`), not concrete classes.
Primitive obsession, Magic numbers	Enums and value objects (`ZoneCommand`, `ComplaintSeverity`, `ComplaintRoutingResult`) and named constants replace raw strings and literals.
Swallowed exceptions	Errors propagate as typed exceptions (`DslSyntaxException`, `AEADBadTagException`); fallback paths are recorded.

7.3 Automated Test Suite and Quality Verification

`ComprehensiveTestSuite` is written as a standalone, zero-dependency test runner that runs directly on any clean JDK 17 setup without requiring external test frameworks like JUnit. It connects to the web Test Center dashboard, running all 35 automated tests across our five pattern implementations with a verified 100% pass rate.

Pattern	Tests	Passed	Main behaviours verified
Decorator	6	6	Pipeline order, decryption, HMAC rejection, corrupted data, invalid key
Composite	8	8	Cascade, restore, aggregation, leaf safety, cycle rejection
Chain of Responsibility	8	8	Severity routing, safe triage, fallback, audit trail
Flyweight	5	5	Instance sharing, thread safety, 72.2% layout saving
Interpreter	8	8	AST, BETWEEN, nesting, missing variables, malformed syntax
Total	**35**	**35**	**100% pass rate**

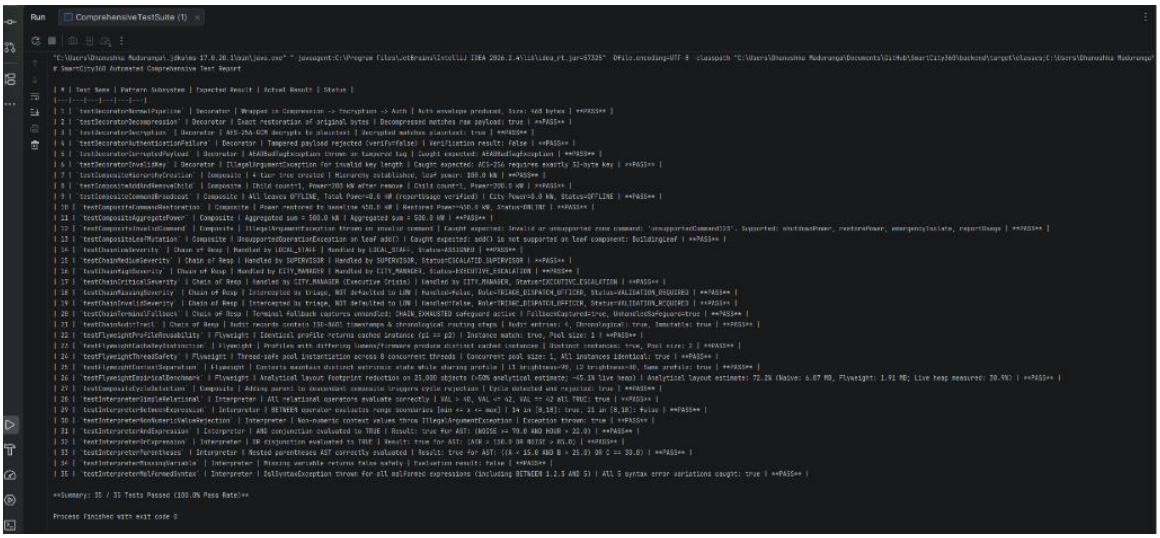


Figure 7.6: Live execution of ComprehensiveTestSuite in IntelliJ IDEA (35/35 passing)

8. Conclusion and References

8.1 Conclusion

In SmartCity360, each Gang of Four design pattern solved a specific municipal software problem: Decorator gives us stackable stream security, Composite manages the urban spatial hierarchy, Chain of Responsibility handles multi-stage ticket escalation, Flyweight cuts heap usage across 25,000 streetlights, and Interpreter provides safe evaluation of dynamic municipal bylaws. Just as importantly, each pattern was applied with an awareness of its trade-offs, from wrapper ordering in Decorator to recursion depth in Composite and class counts in Interpreter. Our automated suite of 35 tests verifies that the entire architecture performs reliably.

For future work, three improvements stand out: migrating the DSL parser to ANTLR if city rules grow beyond Boolean comparisons, adding a distributed cache like Redis to sync flyweight profiles across multiple server nodes, and adding digital signatures (like ECDSA) for citizen grievance non-repudiation.

8.2 References

Coward, D. (2017). Java Servlet Specification, Version 4.0 (JSR 369). Oracle and Eclipse Foundation.

Dworkin, M. (2007). Recommendation for Block Cipher Modes of Operation: GCM and GMAC. NIST SP 800-38D.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Goetz, B. et al. (2006). Java Concurrency in Practice. Addison-Wesley.

Krawczyk, H., Bellare, M. and Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. RFC 2104. IETF.

Martin, R. C. (2008). Clean Code. Prentice Hall.

Oracle. (2024). Java SE 17 Developer Documentation and HotSpot Memory Management Guide.

Appendix A: Source Code Screenshot Catalog

Working source files are grouped by pattern below. The complete compilable project is in the submitted ZIP.

A.1 Decorator (com.smartcity.pattern.decorator)



```
package com.smartcity.pattern.decorator;

public interface StreamData {

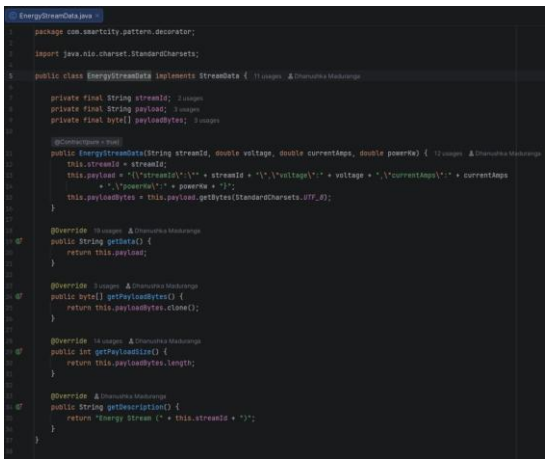
    String getData();

    byte[] getPayloadBytes();

    int getPayloadSize();

    String getDescription();
}
```

StreamData.java (component interface)



```
package com.smartcity.pattern.decorator;

import java.nio.charset.StandardCharsets;

public class EnergyStreamData implements StreamData {

    private final String streamId;
    private final String payload;
    private final byte[] payloadBytes;

    public EnergyStreamData(String streamId, double voltage, double currentAmps, double powerKw) {
        this.streamId = streamId;
        this.payload = "{\"streamId\":\"" + streamId + "\",\"voltage\":" + voltage + ",\"currentAmps\":" + currentAmps
                + ",\"powerKw\":" + powerKw + "}";
        this.payloadBytes = this.payload.getBytes(StandardCharsets.UTF_8);
    }

    @Override
    public String getData() {
        return this.payload;
    }

    @Override
    public byte[] getPayloadBytes() {
        return this.payloadBytes.clone();
    }

    @Override
    public int getPayloadSize() {
        return this.payloadBytes.length;
    }

    @Override
    public String getDescription() {
        return "Energy Stream (" + this.streamId + ")";
    }
}
```

EnergyStreamData.java (concrete component)



```
package com.smartcity.pattern.decorator;

public abstract class StreamDataDecorator implements StreamData {

    protected final StreamData wrappee;

    public StreamDataDecorator(StreamData wrappee) {
        if (wrappee == null) {
            throw new IllegalArgumentException("Wrappee component cannot be null");
        }
        this.wrappee = wrappee;
    }

    public StreamData getWrappee() {
        return this.wrappee;
    }

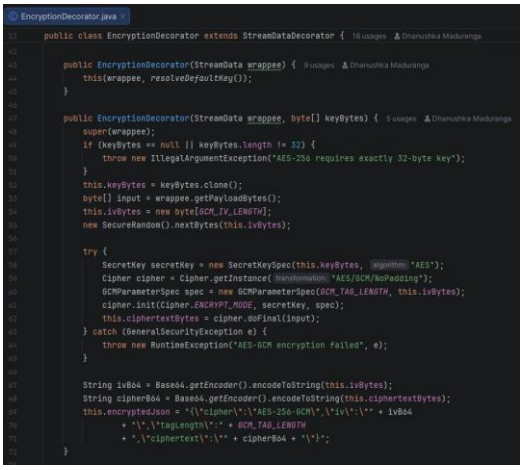
    @Override
    public String getData() {
        return this.wrappee.getData();
    }

    @Override
    public byte[] getPayloadBytes() {
        return this.wrappee.getPayloadBytes();
    }

    @Override
    public int getPayloadSize() {
        return this.wrappee.getPayloadSize();
    }

    @Override
    public String getDescription() {
        return this.wrappee.getDescription();
    }
}
```

StreamDataDecorator.java (abstract decorator)



```
public class EncryptionDecorator extends StreamDataDecorator {

    public EncryptionDecorator(StreamData wrappee) {
        this(wrappee, resolveDefaultKey());
    }

    public EncryptionDecorator(StreamData wrappee, byte[] keyBytes) {
        super(wrappee);
        if (keyBytes == null || keyBytes.length != 32) {
            throw new IllegalArgumentException("AES-256 requires exactly 32-byte key");
        }
        this.keyBytes = keyBytes.clone();
        byte[] input = wrappee.getPayloadBytes();
        this.ivBytes = new byte[GCM_IV_LENGTH];
        new SecureRandom().nextBytes(this.ivBytes);

        try {
            SecretKey secretKey = new SecretKeySpec(this.keyBytes, "AES");
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            GCMParameterSpec spec = new GCMParameterSpec(GCM_TAG_LENGTH, this.ivBytes);
            cipher.init(Cipher.ENCRYPT_MODE, secretKey, spec);
            this.ciphertextBytes = cipher.doFinal(input);
        } catch (GeneralSecurityException e) {
            throw new RuntimeException("AES-GCM encryption failed", e);
        }

        String ivB64 = Base64.getEncoder().encodeToString(this.ivBytes);
        String cipherB64 = Base64.getEncoder().encodeToString(this.ciphertextBytes);
        this.encryptedJson = "{\"cipher\":\"AES-256-GCM\",\"iv\":\"" + ivB64
                + "\",\"tagLength\":" + GCM_TAG_LENGTH
                + ",\"ciphertext\":\"" + cipherB64 + "\"}";
    }
```

EncryptionDecorator.java (AES-256-GCM)



```
package com.smartcity.pattern.decorator;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.zip.GZIPInputStream;
import java.util.zip.GZIPOutputStream;

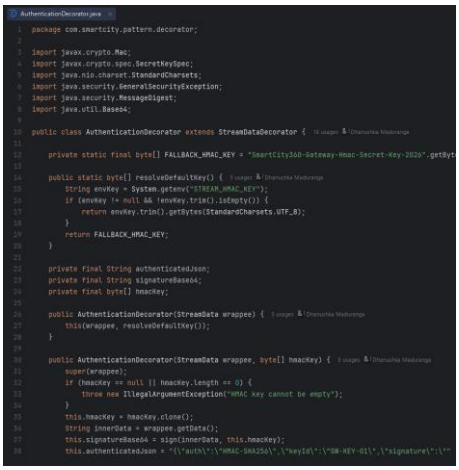
public class CompressionDecorator extends StreamDataDecorator {

    private final String compressedJson;
    private final byte[] compressedBytes;
    private final int originalSize;
    private final int compressedSize;

    public CompressionDecorator(StreamData wrappee) {
        super(wrappee);
        byte[] input = wrappee.getPayloadBytes();
        this.originalSize = input.length;
        this.compressedBytes = compressBytes(input);
        this.compressedSize = this.compressedBytes.length;
        String b64 = Base64.getEncoder().encodeToString(this.compressedBytes);
        this.compressedJson = "{\"format\":\"GZIP\",\"originalBytes\":" + this.originalSize
                + ",\"compressedBytes\":" + this.compressedSize
                + ",\"data\":\"" + b64 + "\"}";
    }

    private static byte[] compressBytes(byte[] data) {
        try {
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            try (GZIPOutputStream gzip = new GZIPOutputStream(baos)) {
                gzip.write(data);
                gzip.finish();
            }
            return baos.toByteArray();
        } catch (IOException e) {
```

CompressionDecorator.java (GZIP)



```
package com.smartcity.pattern.decorator;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.GeneralSecurityException;
import java.security.MessageDigest;
import java.util.Base64;

public class AuthenticationDecorator extends StreamDataDecorator {

    private static final byte[] FALLBACK_HMAC_KEY = "SmartCity360-Gateway-Hmac-Secret-Key-2026".getByt

    public static byte[] resolveDefaultKey() {
        String envKey = System.getenv("STREAM_HMAC_KEY");
        if (envKey != null && !envKey.trim().isEmpty()) {
            return envKey.trim().getBytes(StandardCharsets.UTF_8);
        }
        return FALLBACK_HMAC_KEY;
    }

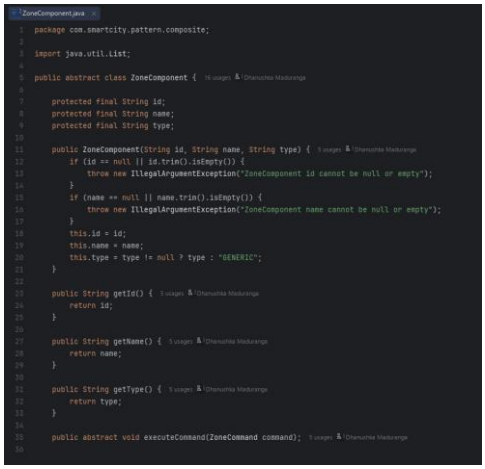
    private final String authenticatedJson;
    private final String signatureBase64;
    private final byte[] hmacKey;

    public AuthenticationDecorator(StreamData wrappee) {
        this(wrappee, resolveDefaultKey());
    }

    public AuthenticationDecorator(StreamData wrappee, byte[] hmacKey) {
        super(wrappee);
        if (hmacKey == null || hmacKey.length == 0) {
            throw new IllegalArgumentException("HMAC key cannot be empty");
        }
        this.hmacKey = hmacKey.clone();
        String innerData = wrappee.getData();
        this.signatureBase64 = sign(innerData, this.hmacKey);
        this.authenticatedJson = "{\"auth\":\"HMAC-SHA256\",\"keyId\":\"SM-KEY-01\",\"signature\":\""
```

AuthenticationDecorator.java (HMAC-SHA256)

A.2 Composite (com.smartcity.pattern.composite)



ZoneComponent.java



BuildingLeaf.java

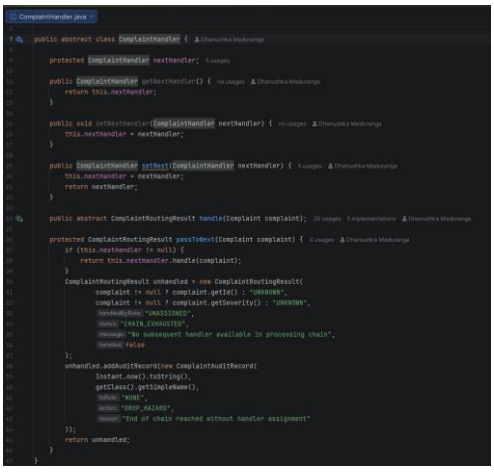


ZoneComposite.java

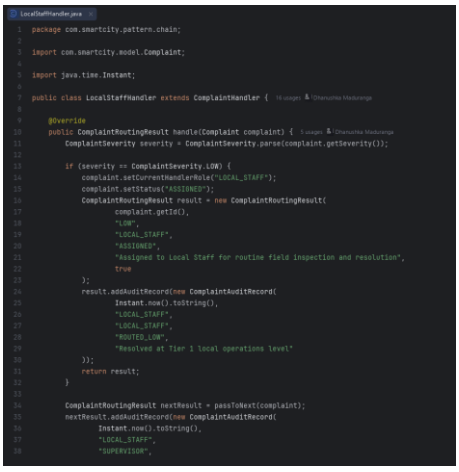


ZoneCommand.java

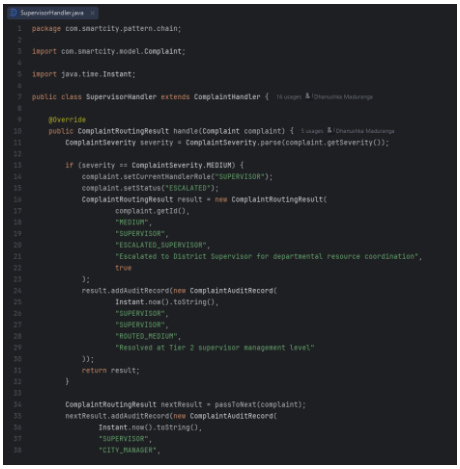
A.3 Chain of Responsibility (com.smartcity.pattern.chain)



ComplaintHandler.java



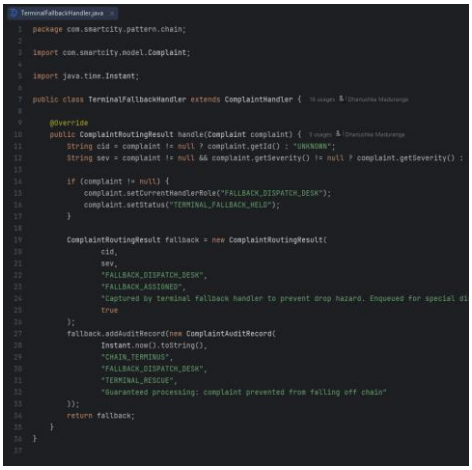
LocalStaffHandler.java



SupervisorHandler.java



CityManagerHandler.java

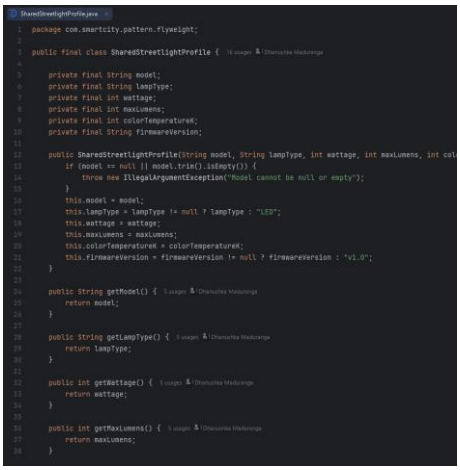


TerminalFallbackHandler.java

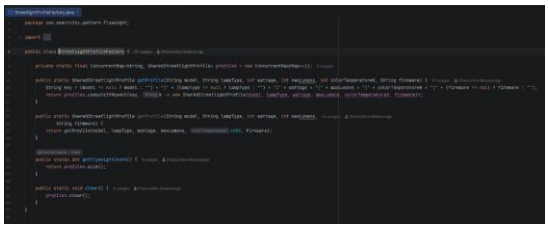


ComplaintChainFactory.java

A.4 Flyweight (com.smartcity.pattern.flyweight)



SharedStreetlightProfile.java



StreetlightProfileFactory.java

```java
package com.smartcity.pattern.flyweight;

public class StreetlightContext {

    private final String poleId;
    private final double latitude;
    private final double longitude;
    private int brightnessLevel;
    private String operationalStatus;
    private final SharedStreetlightProfile profile;

    public StreetlightContext(String poleId, double latitude, double longitude, SharedStreetlightProfi
        if (poleId == null || poleId.trim().isEmpty()) {
            throw new IllegalArgumentException("Pole ID cannot be null or empty");
        }
        if (profile == null) {
            throw new IllegalArgumentException("Shared profile cannot be null");
        }
        this.poleId = poleId;
        this.latitude = latitude;
        this.longitude = longitude;
        this.profile = profile;
        this.brightnessLevel = 0;
        this.operationalStatus = "ACTIVE";
    }

    public void setBrightnessLevel(int level) {
        this.brightnessLevel = Math.max(0, Math.min(100, level));
    }

    public void setOperationalStatus(String status) {
        this.operationalStatus = status != null ? status : "ACTIVE";
    }

    public String display() {
        return this.profile.render(poleId, latitude, longitude, brightnessLevel, operationalStatus);
    }
```

StreetlightContext.java

```java
        public long flyweightTimeMs;
        public int flyweightPoolSize;
        public String jvmInfo;
    }

    public static class ScalingComparisonRow {
        public int count;
        public long naiveEstimatedBytes;
        public long flyweightEstimatedBytes;
        public double analyticalSavingsPercent;
        public long naiveAllocatedBytes;
        public long flyweightAllocatedBytes;
        public double measuredHeapSavingsPercent;
        public long naiveTimeMs;
        public long flyweightTimeMs;
    }

    public static List<ScalingComparisonRow> runScalingSuite() {
        int[] scales = {1000, 10000, 25000, 100000};
        List<ScalingComparisonRow> rows = new ArrayList<>();
        for (int scale : scales) {
            BenchmarkResult b = runBenchmark(scale);
            ScalingComparisonRow row = new ScalingComparisonRow();
            row.count = scale;
            row.naiveEstimatedBytes = b.naiveEstimatedBytes;
            row.flyweightEstimatedBytes = b.flyweightEstimatedBytes;
            row.analyticalSavingsPercent = b.analyticalSavingsPercent;
            row.naiveAllocatedBytes = b.naiveAllocatedBytes;
            row.flyweightAllocatedBytes = b.flyweightAllocatedBytes;
            row.measuredHeapSavingsPercent = b.measuredHeapSavingsPercent;
            row.naiveTimeMs = b.naiveTimeMs;
            row.flyweightTimeMs = b.flyweightTimeMs;
            rows.add(row);
        }
        return rows;
    }

    public static BenchmarkResult runBenchmark(int count) {
```

FlyweightBenchmark.java

A.5 Interpreter (com.smartcity.pattern.interpreter)

```java
package com.smartcity.pattern.interpreter;

public interface Expression {
    boolean interpret(PolicyContext context);
}
```

Expression.java

```java
package com.smartcity.pattern.interpreter;

import java.util.Set;

public class TerminalExpression implements Expression {

    private static final Set<String> VALID_OPERATORS = Set.of(">", ">=", "<", "<=", "==", "!=");
    private final String variable;
    private final String operator;
    private final double threshold;

    public TerminalExpression(String variable, String operator, double threshold) {
        if (variable == null || variable.trim().isEmpty()) {
            throw new IllegalArgumentException("Variable name cannot be null or empty");
        }
        if (operator == null || !VALID_OPERATORS.contains(operator.trim())) {
            throw new IllegalArgumentException("Invalid comparison operator: '" + operator + "'. Suppo
        }
        this.variable = variable.trim();
        this.operator = operator.trim();
        this.threshold = threshold;
    }

    public String getVariable() {
        return variable;
    }

    public String getOperator() {
        return operator;
    }

    public double getThreshold() {
        return threshold;
    }

    @Override
    public boolean interpret(PolicyContext context) {
        if (context == null || !context.has(this.variable)) {
```

TerminalExpression.java

```java
package com.smartcity.pattern.interpreter;

public class AndExpression implements Expression {

    private final Expression left;
    private final Expression right;

    public AndExpression(Expression left, Expression right) {
        if (left == null || right == null) {
            throw new IllegalArgumentException("Sub-expressions in AndExpression cannot be null");
        }
        this.left = left;
        this.right = right;
    }

    public Expression getLeft() {
        return left;
    }

    public Expression getRight() {
        return right;
    }

    @Override
    public boolean interpret(PolicyContext context) {
        return this.left.interpret(context) && this.right.interpret(context);
    }

    @Override
    public String toString() {
        return "(" + left + " AND " + right + ")";
    }
}
```

AndExpression.java

```java
package com.smartcity.pattern.interpreter;

public class OrExpression implements Expression {

    private final Expression left;
    private final Expression right;

    public OrExpression(Expression left, Expression right) {
        if (left == null || right == null) {
            throw new IllegalArgumentException("Sub-expressions in OrExpression cannot be null");
        }
        this.left = left;
        this.right = right;
    }

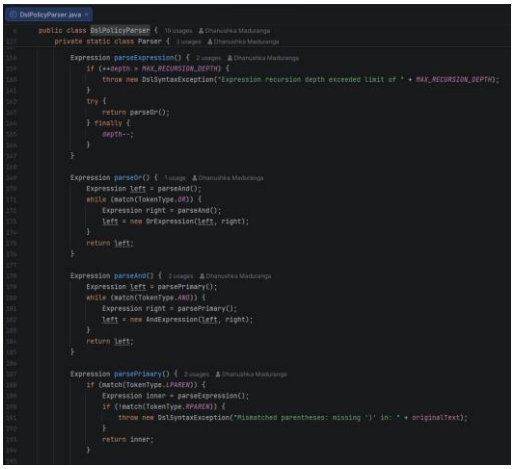
    public Expression getLeft() {
        return left;
    }

    public Expression getRight() {
        return right;
    }

    @Override
    public boolean interpret(PolicyContext context) {
        return this.left.interpret(context) || this.right.interpret(context);
    }

    @Override
    public String toString() {
        return "(" + left + " OR " + right + ")";
    }
}
```

OrExpression.java



DslPolicyParser.java



PolicyContext.java